# EE4C12 WICOS — Course Models & Training Guideline

Oct 9, 2026 · @Huib

## Models taught

The course teaches linear models (regression, logistic regression, SVM), neural networks (MLP, CNN, GNN) and two unsupervised tools (k-means, PCA). Trees, random forests, boosting and kNN are not taught. Page numbers refer to the lecture PDFs in `EE4C12_Course_Materials`.

| Model | Lecture | What the slides cover | Key hyperparameters |
|---|---|---|---|
| K-means | L1 p56–60 | Clustering by alternating assignment and centroid updates; on the practice exam | k |
| PCA, PCA-sphering | L1 p55, L4 p6–13 | Visualisation, whitening (centre, rotate, scale) and dimensionality reduction | Number of components |
| Linear regression | L2 p6–34 | Least squares via the normal equation or gradient descent; Newton's method compared | Learning rate |
| Ridge (L2), Lasso (L1) | L2 p43–47 | Shrinkage; Lasso sets coefficients to zero, so it selects features | λ |
| Weighted and multi-output regression | L2 p48–53 | Per-sample weights; several targets at once | — |
| Logistic regression | L3 p8–11, p25–47 | Sigmoid and cross-entropy; multi-class via one-vs-all or softmax | Threshold, regularisation |

| Model | Lecture | What the slides cover | Key hyperparameters |
|---|---|---|---|
| Support vector machine | L3 p12–18 | Hard and soft margin, hinge loss, C = 1/(2λ); RBF kernel on one starred slide | C, kernel γ |
| Nonlinear features, universal approximators | L4 p15–43 | Fixed-shape (polynomial), neural-network and tree units feeding a linear model | Number of units B |
| Bagging of cross-validated models | L4 p66–67 | Average the models trained on several train/validation splits | Number of splits |
| Multilayer perceptron (MLP) | L5 p12–51 | Single neuron = logistic regression; width vs depth; sklearn `MLPClassifier`, Keras or PyTorch | Width, depth, activation, learning rate, batch size, L2, dropout |
| Convolutional neural network (CNN) | L7 p49–56 | Convolution with stride, max pooling, translation equivariance | Filters, kernel size, stride, pooling |
| Graph neural network, DeepSets | L7 p27–47 | Permutation invariance; GCN, GAT and message passing | — |
| Recurrent neural network | L7 p57 | One slide only | — |

Not taught: decision trees, random forest and gradient boosting (book chapter 14 is not ticked), kNN (only an example of a hyperparameter, L6 p73), naive Bayes, LDA, LSTM and transformers.

## Skills taught

The skills that carry the project are the Lecture 6 workflow, cross-validation with hyperparameter search, regularisation, and an sklearn `Pipeline` that keeps test data out of every fitted step.

| Area | What the course teaches | Use in this project |
|---|---|---|
| Pre-processing | Cleaning (missing values, noise, outliers), mean imputation, min-max and standard scaling (L6 p8–14) | Done in `prepare_data.py`; scale features inside each fold |
| Feature engineering | Nonlinear feature transforms (L4 p15–24); histogram and edge features (L6 p16–30) | 27 centroid, envelope and energy features in `X_feat` |
| Feature selection, dimensionality reduction | Lasso (L2 p45), PCA and PCA-sphering (L4 p6–13), “not all features are important” (L6 p32–33) | Compare all features, L1 selection and PCA (15% of the grade) |
| Losses | MSE; binary and categorical cross-entropy; softmax cost (L2, L3, L5) | Cross-entropy for every classifier |
| Optimisation | Gradient descent, learning rate, mini-batch SGD, epochs, backpropagation (L2 p26–32, L5 p36–42); Adam marked out of scope | Mini-batch training for the MLP and CNN; justify Adam if used |
| Regularisation | L1/L2, dropout, early stopping, SGD noise (L2 p43–47, L5 p47–50) | C or λ for linear models; dropout and early stopping for the CNN |
| Bias-variance | Under- and overfitting via model capacity (L1 p54, L4 p50–61, L6 p49–54) | Keep models small: 960 samples |
| Validation | Train/validation/test, overfitting the validation set, naive and k-fold CV (L4 p53–64, L6 p55–68) | Grouped CV by person (see Training guideline) |
| Hyperparameter search | Grid vs random search; random wins when some parameters don’t matter (L6 p73–79) | Grid for 1–2 parameters, random search beyond that |

| Area | What the course teaches | Use in this project |
|---|---|---|
| Metrics | Confusion matrix, accuracy, precision, recall, F1, ROC, balanced and per-class accuracy (L3 p20–48) | Accuracy, macro-F1, confusion matrix |
| Class imbalance | Weighted cross-entropy; balanced accuracy (L3 p35–36) | Not needed: 240 samples per class |
| Pipelines and leakage | Scaling → selection → model in one `Pipeline`, tuned as a whole; test data never part of the pipeline (L6 p71, p80) | Every model is a `Pipeline` |
| Neural-network practice | Activations, initialisation, parameter counts, run-to-run randomness (L5 p9–52) | Report parameter count; repeat with 3 seeds |

Grouped data (several samples per person) is not covered in the slides, so the choice of grouped CV must be argued in the report under “Appropriate use of data” (10%).

## Model shortlist

Optimise four models that climb from linear to deep: softmax logistic regression, an RBF SVM, an MLP and a small CNN. All four are taught, and the CNN covers the mandatory deep network. Each step up the ladder answers one question in the report.

| # | Model | Input | Question it answers | Hyperparameters to tune (search range) |
|---|---|---|---|---|
| 1 | Softmax logistic regression, L2 or L1 | 27 features, standardised | How far do hand-crafted features go with a linear model? L1 also gives feature selection | C: $10^{-3}$–$10^{3}$ (log); penalty: L1, L2 |
| 2 | SVM, RBF kernel | 27 features, standardised | Does a nonlinear boundary on the same features help? | C: $10^{-1}$–$10^{3}$ (log); γ: $10^{-4}$–$10^{1}$ (log); number of features kept |

| # | Model | Input | Question it answers | Hyperparameters to tune (search range) |
|---|---|---|---|---|
| 3 | MLP (`MLPClassifier`) | Images reduced by PCA, fitted per fold | Does a dense network on the raw image beat the features, and does convolution then add anything? | PCA components: 20–200; hidden layers: 1–2 of 32–256 units; L2 α: $10^{-5}$–$10^{-1}$; learning rate: $10^{-4}$–$10^{-2}$ |
| 4 | Small CNN (Keras or PyTorch) | 128 × 128 images (`X_img`) | Does learned spatial structure beat hand-crafted features across people? | Learning rate: $10^{-4}$–$10^{-2}$; dropout: 0–0.5; filters in the first block: 8, 16, 32; batch size: 16, 32 |

CNN design: 3 blocks of conv 3 × 3 → ReLU → max-pool 2 × 2 (filters doubling per block), global average pooling, dropout, dense softmax over 4 classes. Keep it under about 100k parameters.

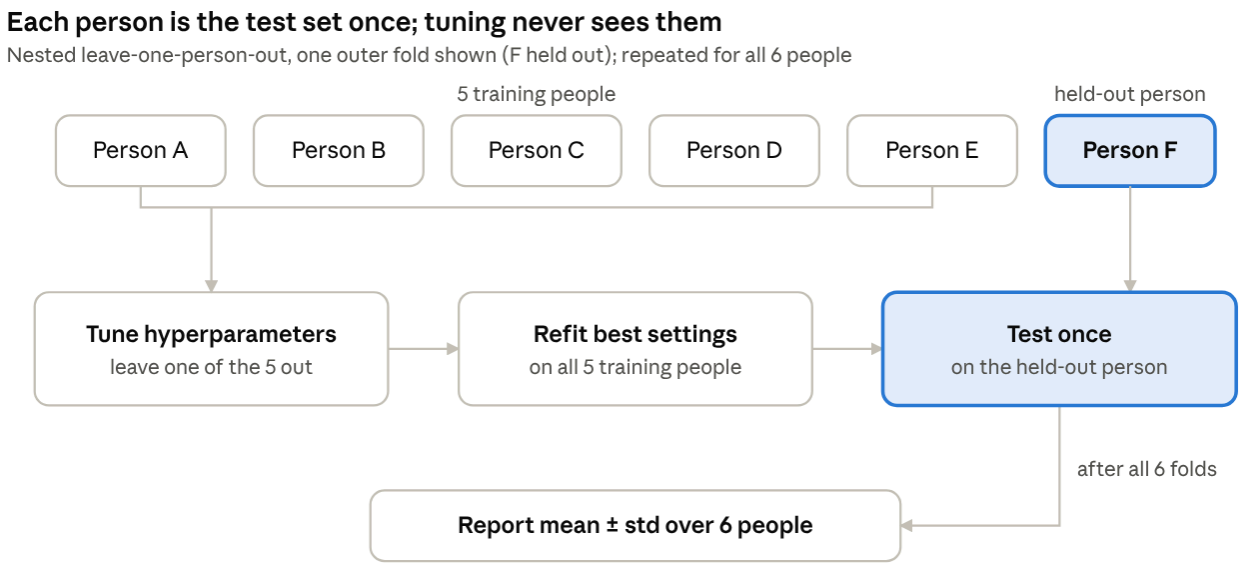
A quick first check on the prepared data (training set only, default settings, run on 9 October) shows the four-model ladder has room to improve, and that people differ a lot:

| Model (default settings) | Random 5-fold accuracy | Leave-one-person-out accuracy |
|---|---|---|
| Logistic regression, features | 0.88 | 0.71 ± 0.14 |
| SVM RBF, features | 0.93 | 0.69 ± 0.17 |
| SVM RBF, images (PCA 50) | 0.94 | 0.60 ± 0.18 |
| Random forest, features | 0.93 | 0.74 ± 0.06 |

Random forest scored best across people but is not taught in the slides. Add it only as an optional fifth model, with one sentence explaining what it is.

## Training guideline

Tune and evaluate every model with nested leave-one-person-out cross-validation, so each score answers the brief's question: how well does it classify a gesture from someone it has never seen? Run all four models on the same folds.




nested leave-one-person-out · one of 6 outer folds

The held-out person is touched only once per fold; every tuning decision comes from the other five.

1. **Load and fix seeds.** Load `processed/wicos_dataset.npz` (`X_feat`, `X_img`, `y`, `persons`). Set `SEED = 42` for NumPy, every sklearn `random_state`, and TensorFlow or PyTorch.
2. **Set up the folds.** The outer loop is `LeaveOneGroupOut` with `groups = persons`: 6 folds, each holding out one person as the test set. The inner loop is `LeaveOneGroupOut` over the 5 remaining people and is used only for tuning.
   - Why not a random split: the 40 samples per person and gesture come from one recording, so near-duplicates leak across a random split. The quick check above shows random-split accuracy about 20 points higher.
3. **Build each model as a `Pipeline`.** Scaler → feature selection or PCA → classifier (L6 p80). Everything that is fitted (scaler, selection, PCA) is fitted inside the fold, never on held-out people.
4. **Run a baseline.** Default hyperparameters through the outer folds. This is the reference every tuned result is compared with.
5. **Tune the classical models.** `GridSearchCV` (logistic regression) or `RandomizedSearchCV` with about 50 draws (SVM, MLP) on the inner folds, scored on macro-F1. Use the ranges in the shortlist table. Save `cv_results_` for the validation-curve figures.
6. **Tune and train the CNN.**
   - Mini-batch training with cross-entropy, up to 100 epochs, early stopping on validation loss with patience 10 (L5 p47); restore the best weights.

- Augment the training batches only: random time shift, small time stretch, small noise, time and Doppler masking. Never flip the Doppler axis; it reverses the motion direction.
- Tune learning rate, dropout and filters with a random search of 10–15 settings. To save time, use one inner validation person per outer fold instead of all 5.
- Repeat the final configuration with 3 seeds and report the mean.

7. **Analyse feature selection.** With logistic regression and SVM, compare all 27 features, the L1-selected subset, the top-k by mutual information (k tuned in the inner loop) and PCA. This fills the 15% feature-selection criterion.
8. **Evaluate on the outer folds.** For each held-out person, refit the best inner-loop configuration on the other 5 people and test once. Report mean ± std over the 6 people.
9. **Report the random split as a contrast.** Score the tuned models once on the notebook's random 80/20 split and explain the gap to leave-one-person-out.
10. **Fit the final model.** Pick the best model by mean outer macro-F1, weighing complexity. Tune it once more with leave-one-person-out over all 6 people and refit on all 960 samples.
11. **Log everything.** Write one CSV row per model, fold and seed (scores, chosen hyperparameters, training time) so every table and figure can be rebuilt.

## Comparison plan

Compare the four models on macro-F1 and accuracy, as mean ± std over the 6 held-out people, and fill in the table below. Macro-F1 is the headline metric: it weighs all four gestures equally and exposes a model that sacrifices pinch or click.

| Model | Accuracy, leave-one-person-out | Macro-F1, leave-one-person-out | Accuracy, random split | Parameters | Training time |
|---|---|---|---|---|---|
| Logistic regression | | | | | |
| SVM RBF | | | | | |
| MLP | | | | | |
| CNN | | | | | |

Figures for the report:

- Validation score against the key hyperparameter of each model (C, γ, PCA components, learning rate), from the inner loop.
- CNN training and validation loss curves, showing where early stopping triggered.

- Confusion matrix of the best model, summed over the 6 outer folds; expect pinch and click to be confused.
- Accuracy per held-out person for all four models in one grouped bar chart, to show which person is hardest.
- Feature-selection results: score against the number of features kept, and the L1 coefficients per gesture.